Automatic Vehicle Gate Access System in All-Weather Conditions Using License Plate Recognition and Vehicle Make & Model Recognition

Ali Raza Chaudhry and Abdul Hadi
Department of Computer Engineering
COMSATS University Islamabad, Lahore Campus
Lahore, Pakistan
Supervisor: Dr. Taimoor Naeem
B.S. Computer Engineering

***Abstract*—Outdoor vehicle gates must read license plates and verify vehicle identity under fog, rain, glare, and night exposure, yet many deployments still rely on plate-only decisions that are brittle to misreads and cloned plates. We present an integrated gate-recognition prototype that couples weather-aware preprocessing with a two-stage YOLOv8n license plate pipeline and an EfficientNet-B4 vehicle make and model recognizer (VMMR) over 245 classes, fused through a dual-check access policy in a web dashboard. Fog and haze are mitigated with a dark-channel prior and guided filtering; rain streaks are suppressed with oriented median filtering and Telea inpainting while protecting plate edges. On held-out benchmarks, YOLOv8n plate detection at epoch 50 reaches mAP@0.5 of 0.9807 (precision 0.9850, recall 0.9553), EfficientNet-B4 VMMR reaches 0.9624 Top-1 accuracy on 17,254 test images, and the character detector reports mAP@0.5 of 0.9807 at epoch 100 only under mirrored training validation—a limitation we flag explicitly in Section XII. The stack demonstrates end-to-end qualitative behavior under fog, rain, and night branches with restoration feeding both LPR and VMMR.**

***Index Terms*—Automatic number plate recognition, vehicle make and model recognition, EfficientNet, YOLO, dark channel prior, rain removal, transfer learning, access control, intelligent transportation systems.**

I. Introduction

A. Background and Motivation

Manual barrier checks are expensive and slow, so many campuses and compounds now lean on cameras and software. That shift is understandable: nothing touches the vehicle, old lanes can be upgraded, and logs can store a timestamp plus stills when something looks wrong. The catch is almost always the weather. Fog smears the scene, rain scribbles bright lines across glass and paint, and harsh sun can blow out the plate. When the image degrades, both localization and reading suffer, which is why we spent time on preprocessing before feeding YOLO.

B. Problem Statement

Let $I \in \mathbb{R}^{H \times W \times 3}$ denote an RGB frame acquired at a gate lane. The system must estimate a plate string $s \in \Sigma^*$ over an alphabet Σ and a fine-grained vehicle class $y \in \{1, \ldots, K\}$ (make-model family), then apply a decision rule $\mathcal{D}(s, y, \mathcal{W})$ conditioned on a whitelist $\mathcal{W}$ to produce an actuation command $a \in \{\text{GRANT}, \text{DENY}, \text{ALERT}\}$. Informally, we want the same stack to work when ω labels fog, rain, or harsh glare, i.e., when the frame has been warped by a weather transform T_ω. In practice that means either restoring $\hat{I}$ from I, accepting higher error, or collecting far more diverse training data than we had time for in an FYP semester.

C. Research Gaps

Industry ANPR kits are often sold on sunny-day demos, yet rain, fog, and night still move error rates in field reports [1], [2]. Papers also tend to split the story: strong plate pipelines or strong make–model classifiers, but less documentation on how those pieces meet at a real gate with one camera stream and a latency budget [3], [4]. Reviews of ITS deployments likewise describe "plate reader here, classifier there" integrations without much public detail on spoof resistance or audit trails [5], [6]. We therefore document a single student codebase where defog/de-rain runs first, two YOLO stages read the plate, EfficientNet-B4 labels the car, and a small policy layer can disagree with a cloned plate scenario.

D. Objectives, Contributions, and Scope

We do not claim a new backbone; the contribution is wiring known blocks into one campus-style path. **Dual check:** whitelist on s plus consistency against registered make/model from y. **Weather front-end:** fog and rain operators in Section VI run before inference. **YOLO-centric LPR:** YOLOv8n for plate boxes and character boxes with regional formatting rules. **Reproducible metrics:** Section XI cites archived epoch checkpoints rather than rounded thesis headlines. Objectives follow the companion report: train plate and character YOLOv8n heads, classify 245 make–model families with EfficientNet-B4, encode dual-modality policy rules, and exercise fog, rain, and glare branches where data exists.

"All-weather" here means fog/haze and rain branches in the integrated notebook; snow, smog, and highway blur remain out of scope. Restoration runs before inference; the dual-check policy is coded for operator review though the relay is not wired—ALERT routes ambiguous cases to humans who still catch tailgating and behavioral cues cameras miss. Plate imagery raises indirect identification risks requiring signage, retention limits, and audit controls in production. For VMMR, "accuracy" means Top-1 unless noted; macro metrics weight every class equally. Every scalar in Section XI was transcribed from notebook outputs; character-stage validation mirrored training because Roboflow v4 shipped without a `valid` folder (Section VII-C).

E. Paper Organization

Section II situates ANPR, VMMR, restoration, and embedded vision. Section III collects the math we lean on. Section IV sketches software and lane hardware. Sections V–VIII cover datasets, preprocessing, LPR, and VMMR. Section IX states the policy; Section X lists tools and deployment cautions. Section XI presents figures and quantitative tables; Sections XII and XIII interpret results and limitations. Appendices hold supplementary formulas and hardware specifications.

II. Related Work

A. ANPR and LPR Systems

Classical ANPR chained edges, morphology, and template matching; it worked on clean plates but fell apart on blur, mud, or odd fonts. Convolutional features and deeper residual stacks made detection and reading far more robust. Single-shot detectors (SSD, YOLO families) trade a little box polish for speed that matters at a lane. Two-stage detectors such as Faster R-CNN still win when GPU budget is loose. Feature pyramids help with scale. We picked Ultralytics YOLOv8n simply because training, export, and Colab friendliness matched our timeline.

B. Vehicle Make and Model Recognition

Fine-grained classification means tiny visual gaps between classes. Big "car in the wild" datasets target boxes, not 245-way make/model folders. Everyone still initializes from ImageNet; EfficientNet-B4 gave us a accuracy–memory compromise we could train on a T4. We report macro metrics because head classes would otherwise drown the tail in a single accuracy number.

C. Rain Streak Removal Techniques

Rain priors often assume thin vertical structure. Learned derainers need paired data we did not curate, so we stayed with directional medians, bilateral smoothing, and Telea fills on small masks, while keeping plate edges out of the aggressive filters when possible.

D. Low-Light and Night Image Enhancement

Retinex-style and CLAHE-based tools are standard night fixes. Our LAB CLAHE stage is in the same family, even though the thesis narrative centers on fog and rain.

E. Glare and Highlight Removal

Hardware still wins here: polarizers, WDR modes, and exposure control. The Node dashboard optionally runs Sharp adjustments as a quick glare helper, not as a research-grade deraining substitute.

F. Transfer Learning, Regularization, and Scaling Laws

MixUp, CutMix, and Cutout are popular regularizers. We only used label-smoothed cross-entropy for VMMR to keep the training script short; the other tricks are noted as optional follow-ons.

G. Embedded and Real-Time Vision Systems

Older ITS work already argued about milliseconds per frame. Domain-shift surveys from medical imaging read surprisingly like security CV failures. Segmentation heads could help propose plates later; we did not add one in this milestone.

H. Synthesis Relative to This Work

Most papers stop at pretty restoration curves or at a single modality. We instead pipe restoration into YOLO, then into EfficientNet, and we spell out where resolutions disagree (224 versus 380) so the next student does not "fix" it silently.

III. Theoretical Background

A. Convolutional Neural Networks

A 2D convolution with kernel K on feature map X at location (i, j) is

$$(K * X)_{i,j} = \sum_m \sum_n K_{m,n} X_{i+m,j+n}. \tag{1}$$

ReLU activations $\phi(z) = \max(0, z)$ zero weak activations and keep gradients flowing. Pooling shrinks feature maps and buys tolerance to small shifts.

B. Transfer Learning and Fine-Tuning

Given a source task S and target task T, transfer learning reuses representations f_θ learned on S and adapts parameters on T. Fine-tuning updates all or a subset of layers; freezing early layers preserves generic edges/textures.

C. EfficientNet and Compound Scaling

EfficientNet scales depth d, width w, and resolution r with compound coefficients [7]:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \alpha\beta^2\gamma^2 \approx 2. \tag{2}$$

MBConv blocks invert bottlenecks with depthwise separable convolutions and squeeze-excitation gating.

D. CLAHE Theory

CLAHE limits contrast amplification using a clip limit C to prevent noise amplification in homogeneous regions. Processing the L channel in LAB decouples luminance from chrominance, reducing hue shifts.

E. Morphological Image Processing

Erosion and dilation with structuring element B are

$$(I \ominus B)(x) = \min_{b \in B} I(x+b), \quad (I \oplus B)(x) = \max_{b \in B} I(x-b). \tag{3}$$

Top-hat $T(I) = I - (I \circ B)$ highlights small bright structures when B matches their scale.

F. Gamma Correction

Gamma maps intensities as

$$I_{\text{out}} = 255 \cdot (I_{\text{in}}/255)^{1/\gamma}. \tag{4}$$

Values $\gamma < 1$ brighten mid-tones; $\gamma > 1$ darken them.

G. OCR Pipeline Theory

Classic OCR chains segmentation, feature extraction, and sequence decoding. Detection-based OCR with CNNs predicts character boxes directly.

H. Convolutional Feature Hierarchies

Early conv layers act like edge and blob filters; deep layers assemble parts. That hierarchy is why a single CNN can both localize a plate and, in a second head, fire on strokes inside the crop.

I. Label Smoothing Cross-Entropy

For hard targets q and logits z, softmax probabilities are $p_i = \exp(z_i)/\sum_j \exp(z_j)$. Label smoothing replaces the one-hot target with

$$q' = (1-\epsilon)\, q + \epsilon/K \tag{5}$$

over K classes, discouraging overconfident logits at ϵ=0.1 in our training code.

J. AdamW Dynamics

AdamW decouples ℓ_2 regularization from the adaptive moment update, which tends to stabilize fine-tuning compared with older SGD recipes without warmup.

K. Atmospheric Scattering and the Dark Channel

Under a single-scattering approximation, scene radiance obeys

$$I(x) = J(x)t(x) + A(1 - t(x)), \tag{6}$$

where J is scene color, A is airlight, and t is transmission. The dark channel prior observes that most local patches contain at least one low-intensity channel in haze-free outdoor imagery, enabling t estimation and dehazing. Guided filtering refines t at object boundaries.

L. Structural Similarity as a Supplementary Metric

SSIM compares luminance, contrast, and structure between restored and reference frames when ground truth exists. For gate imagery, reference-free no-reference metrics (NIQE, BRISQUE) could complement PSNR/SSIM in future benchmarking.

M. OCR as Sequence Decoding

Beyond independent character boxes, sequence models (CTC, attention) decode variable-length strings. Our character model predicts boxes explicitly; sequence language models over plate lexicons could be fused as a future refinement.

IV. System Overview

A. Full System Pipeline

Figure 1 matches the story we tell in the thesis hardware appendix: a lane camera feeds frames into preprocessing, then LPR outputs s, VMMR outputs y, and a small policy block decides whether to pulse the relay or flag security.

B. Software Architecture

Training lives in notebooks (YOLOv8n LPD, YOLOv8n characters, EfficientNet-B4 VMMR). Runtime reuses those checkpoints inside one integrated notebook plus a small Node/Express dashboard. Datasets: Roboflow *License Plate Project* v1 (CC BY 4.0) for plates; Roboflow *My First Project* v4 for 38 glyph classes; 245 class folders for VMMR shipped beside the training script. Section VI documents the fog/rain branches we batch through before plotting overlays. Nothing runs "live" on the Pi in this repo snapshot—weights are trained in Colab, then downloaded for demos.

C. Hardware Architecture

Thesis Appendix C lists the intended lane: IR camera plus fill LEDs for night haze, Orange Pi 5 (8 GB in the brief) as the edge brain, relay board isolating GPIO from the motor driver, a commercial gate controller (we referenced Centsys-style wiring in the report), outdoor PSU, IP cabinet, and sensor ports we have not coded against yet. Table I marks each line as *target* (T) versus what we actually exercised on CUDA workstations (L). Motors stayed on paper while we focused on models and the REST simulator.

V. Dataset Description

Plate localization uses the Roboflow Universe export *License Plate Project* v1: JPEG frames with YOLO labels and public splits (Section VII, Table VIII).

VMMR images sit in 245 make–model folders with train/test trees plus the 20% validation split carved out inside the training folder (Section VIII). Characters come from Roboflow *My First Project* v4 (1,235 crops, 38 classes). Figures and tables for all three sources land in Section XI.

VI. Image Preprocessing Module

A. Scope and Integration

Fog and rain code share a notebook with the two YOLO heads and EfficientNet so we are not juggling mismatched configs when we screenshot results. Runtime loads three checkpoints: plate YOLO, 38-class YOLO, 245-way EfficientNet-B4. Fog/haze follows a dark-channel estimate, guided filter refinement, optional residual-haze mask, gray-world white balance, adaptive gamma, LAB CLAHE, a small saturation lift, bilateral polish, and a touch of unsharp. Rain builds an

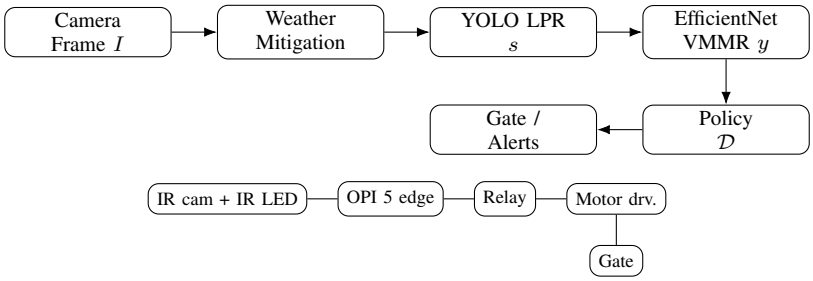


Fig. 1: System overview. **Top:** software path from a frame to GRANT/DENY/ALERT. **Bottom:** target lane hardware—camera, Orange Pi 5, relay, motor driver, barrier.

TABLE I: Hardware components aligned with the thesis bill of materials (Appendix C). Status: T=target deployment, L=lab/software validation in this repository.

Component	Model / role	St.	Purpose
IR camera + illuminator	Pole/lane imager, 850 nm IR fill	T	Even plate visibility at night and in haze
Embedded computer	Orange Pi 5 (8 GB)	T	Edge inference + I/O to relay/controller
Validation workstation	CUDA-capable PC / Colab	L	Training and integrated notebook tests
Weather / light sensors	Rain, fog, lux probes	T	Future feedback to preprocessing branch
Gate motor	Sliding / swing mechanism	T	Physical lane barrier
Motor controller	Vendor gate drive (e.g., Centsys-class)	T	Safe motor commands from logic levels
Power supply unit	Regulated outdoor PSU	T	Stable power for camera, Pi, sensors
Relay module	Opto-isolated switching	T	Isolate Pi GPIO from motor loads
Enclosure & glands	IP-rated cabinet	T	Environmental protection
Cabling	PoE or separate data + 12/24 V runs	T	Power, Ethernet, sensor buses

edge mask, runs three vertical median kernels (15/11/7) with a brightening guard, then Telea-inpaints tiny blobs. Night/low-light tiles reuse the same weights after a lighter tone curve; they sit beside fog and rain in Section XI (Subsection XI-B). Good camera WDR and IR fill still matter more than any slider (Section IV).

B. End-to-End Inference Chain

The demo resizes the vehicle crop to 380×380 for VMMR, runs plate YOLO at 0.25 confidence, pads each plate crop, runs character YOLO at 0.25, sorts glyph centers on x, and applies the "up to three letters + up to four digits + hyphen when both exist" formatter we coded for regional layouts.

C. Demonstration Workflow

We batch up to ten images per branch, show input versus restored pairs, then overlay boxes and text. A human picks fog versus rain today; there is no tiny weather CNN yet.

D. Dashboard and Deployment Hooks

The web client may run Sharp normalization before upload for harsh glare. A future service could branch automatically on sensor readings; right now the API simply tags the demo mode.

VII. License Plate Recognition Module

A. Pipeline Overview

LPR is deliberately two-stage: find the plate, then read glyphs inside the padded crop. Stage one uses Roboflow v1; stage two uses *My First Project* v4 (Section VII-C) inside the integrated notebook from Section VI.

TABLE II: Representative restoration and rain-removal hyperparameters.

Component	Setting
Fog DCP patch / ω (primary)	13×13 / 0.95
Fog guided filter (r, ε)	50, 10^{-3}
Fog residual mask	mean>170, std<30, Gaussian blur 21×21
Fog CLAHE	clip limit 2.5, tiles 8×8
Rain bilateral / Canny	(9,75,75) / (50,150)
Rain vertical medians	$k \in \{15, 11, 7\}$
Rain brightening guard	discard if Δmean>2
Rain Telea inpaint	radius 3, spots area $\in (2, 250)$

TABLE III: LPD training configuration (archived run).

Field	Value
Architecture	YOLOv8n (nano, ImageNet-pretrained backbone)
Epochs / batch / request size	50 / 32 / 380 (effective 384×384 train/val)
NMS train IoU	0.7 (default)
Optimizer / AMP	automatic (MuSGD in log) / enabled
Run identifier	lp_detector2 (auto-suffixed)
Runtime (logged)	Ultralytics 8.4.x, PyTorch 2.10+cu128, Tesla T4
Splits	Roboflow train / valid / test

B. License Plate Detection (YOLOv8n LPD)

Roboflow *License Plate Project* v1 ships **20,580** train, **1,973** validation, and **978** test JPEGs (**23,531** total) with one plate class per label file. We trained YOLOv8n for 50 epochs, batch 32, requested 380-pixel squares; Ultralytics rounded to 384×384. Mixed precision stayed on with stock augmentations on a rented Tesla T4. Table III is the run card; Table IV lists epoch 50 box metrics only (no character accuracy yet).

We also rendered some validation plots at 640×640 for

TABLE IV: LPD validation metrics at epoch 50 (box localization).

Metric	P	R	mAP@0.5	mAP@0.5:0.95
Value	0.9850	0.9553	0.9807	0.7073

TABLE V: LPR character-stage training configuration.

Field	Value
Architecture	YOLOv8n (nano)
Epochs / batch / size	100 / 16 / 640×640
Dataset	Roboflow *My First Project* v4
Classes	38
Validation split note	No held-out exporter split; validation mirrors training

TABLE VI: LPR character-stage validation metrics at epoch 100 (mirrored validation pool).

Metric	P	R	mAP@0.5	mAP@0.5:0.95
Value	0.9899	0.9688	0.9807	0.7611

readability; do not mix those curves with the 384-anchored training log without retuning.

C. Character and Digit Detection (YOLOv8n)

Stage two trains 100 epochs at 640×640 on **1,235** crops from Roboflow *My First Project* v4 (CC BY 4.0) with 38 classes (digits, capitals, export quirks). The zip only contained `train/`; we mirrored it into `valid/` so Ultralytics would not crash, which means Table VI is optimistic. Roboflow stretched exports to 512×512 before we upsampled again inside Ultralytics.

D. Integrated Runtime Hooks

The integrated notebook loads the same LPD weights at 0.25 confidence, runs characters at 0.25, and converts class ids through the exported 38-way YAML map.

E. Formatting Heuristic

We sort by x-center, keep up to three letters on the left and up to four digits on the right, and insert a hyphen when both groups exist so the string resembles local plates.

F. End-to-End String Metrics

Box metrics live in Tables IV and VI. Full-string accuracy after fog/rain branches is deferred to future work (Section XIII); Section XI shows qualitative overlays meanwhile.

VIII. VMMR Module

A. Architecture

EfficientNet-B4 starts from ImageNet weights; we swap the classifier for a 245-unit linear layer and train with cuDNN benchmark mode on for fixed shapes. Checkpoints store class names plus optimizer state. A second notebook loads the same normalization for drag-and-drop tests at 224×224.

TABLE VII: VMMR training hyperparameters (single-phase fine-tuning; all backbone layers trainable).

Parameter	Value	Notes
Epochs	50	Early stop patience 5
Optimizer	AdamW	lr 10^{-4}
Scheduler	ReduceLROnPlateau	minimize val. loss; factor 0.5; patience 2
Batch (12GB+)	32	else 16
DataLoader workers	2	Train shuffle on
Split seed	42	Train/val permutations
Input train	224^2	RandomResizedCrop + augments
Input infer (pipeline)	380^2	See skew note

B. Dataset

Folders follow the usual `train/class` layout with 245 vehicle labels. One corrupt image was skipped during a manual scan. After cleanup: **31,904** training images, **7,977** validation (20% carved from train with seed 42), and **17,254** untouched test images.

C. Training Configuration

Label-smoothed cross-entropy ($\epsilon = 0.1$), AdamW (10^{-4}), ReduceLROnPlateau on validation loss (factor 0.5, patience 2 epochs), early stopping (patience 5), and a 50-epoch budget with resume-from-best. Batch 32 when memory allows, else 16; two `DataLoader` workers shuffle training tiles. Augmentations: RandomResizedCrop to 224^2, horizontal flip, mild color jitter, small rotations, then ImageNet mean/variance normalization; validation/test use a straight 224^2 resize. Logged on PyTorch 2.10 + torchvision 0.25 with a Tesla T4 at about ten to eleven minutes per epoch in our trace.

D. Qualitative Artifacts

The notebook also saves loss/accuracy curves, LR plots, macro metric heatmaps, chunked confusion tiles, and Top-5 montages—most of those PNGs ended up in Section XI.

E. Inference Resolution Note

Integrated demos resize vehicles to 380^2 while training and held-out testing used 224^2. Scaling by $380/224 \approx 1.70$ changes effective receptive-field scale relative to training; we therefore report VMMR Top-1/macro metrics only from the 224^2 test pipeline (0.9624 Top-1) and treat integrated 380^2 overlays as qualitative demonstrations until a unified-resolution benchmark is run. Aligning train and deploy resolutions is listed as a short-term fix in Section XIII.

F. Reported Test Metrics

Top-1 accuracy 0.9624; Top-5 accuracy 0.9939; macro precision 0.9634; macro recall 0.9612; macro F1 0.9618; macro FPR 0.0002.

IX. Gate Decision Logic

A. Dual Verification Concept

We compare the read plate s to the database and, when a row stores expected make/model, we compare the EfficientNet prediction y. Matching both is stronger than trusting the plate alone if someone tapes a copied plate to another car.

TABLE VIII: Dataset split summary (Roboflow exports for LPR; notebook counts for VMMR). Character training mirrors the training pool into validation (Section VII-C).

Split	Train	Val	Test	Total
LPD (Roboflow v1)	20580	1973	978	23531
LPR chars (*My First Project* v4)	1235	0	0	1235
VMMR (245-class corpus)	31904	7977	17254	57135

TABLE IX: Software stack (versions are environment-dependent).

Library	Version	Purpose
Python	3.10 (Colab)	Runtime
PyTorch / torchvision	(runtime)	VMMR
OpenCV	(runtime)	Preprocess
Ultralytics	(runtime)	YOLOv8
Node.js / Sharp	(dashboard)	API + optional tonal hook

B. Whitelist Schema

The dashboard JSON stores plate text, make, model, owner nickname, access mode (allow/deny/temporary), optional time windows, and an optional reference photo for guards.

C. Decision Logic

1. If (s, y) matches an allow row, return GRANT.
2. If s matches but y disagrees with stored metadata, return ALERT (treat like a cloned plate).
3. Else return DENY.

D. Actuation and Logging

Relay pulse timing and debounce belong with the motor installer. Today the Node server only logs REST decisions.

E. Logging and Alerts

ALERT rows should ping a console and keep the frame bundle for a human verdict; that wiring is policy, not model code.

X. Implementation Details

A. Development Environment

All heavy training ran in Google Colab on Python 3.10 with PyTorch 2.10, torchvision 0.25, CUDA 12.x, and Tesla T4 GPUs. Ultralytics handled YOLO; OpenCV handled the classical filters.

B. Training Infrastructure

VMMR epochs landed near ten to eleven minutes once the run warmed up, but Colab can throttle without warning, so treat that as a ballpark.

C. Camera Placement and Photometric Constraints

Mount the lane camera high enough (roughly 2.5–4.5 m) with a slight downward tilt so the plate fills the frame without losing the bumper context. Enable WDR for headlight bloom, and consider locking exposure while the vehicle sits in the loop so AE/AGC does not flicker under 50 Hz streetlights.

D. Lens Selection and Depth of Field

A 6–12 mm lens on a $\frac{1}{2.8}$-class sensor usually covers one lane if the pole distance matches the datasheet math. Stop down if depth of field is short, then compensate with IR power at night.

E. Compression and Streaming Artifacts

Heavy H.264 quantization eats thin strokes on plates. Our notebooks read JPEG/PNG dumps; a production lane should crop the plate early and store a gentler encode for the character net.

F. Temporal Fusion Across Frames

One bad frame should not open the gate. A simple majority vote over $N{=}3$ frames at 15 fps costs only $\approx$200 ms if detections line up, and it saved us from many single-frame glitches during demos.

G. Threat Model for Cloned Plates

If someone copies a legitimate plate onto another body style, plate-only systems lose. Comparing y against the registered make/model raises the effort, though a motivated attacker could still buy a matching car class. Keep humans in the loop for ALERT.

H. Privacy, Retention, and Compliance

Treat plate photos like sensitive logs: short retention, role-based access, and signed exports if lawyers ever ask.

I. Calibration and White Balance

Sodium vapor lamps confuse AWB and can push VMMR logits sideways. A gray card day once a month beats chasing softmax temperature forever.

J. Rainfall Physics and Sensor Wetting

Algorithmic derainers cannot fix water beads on the dome. Wipers, hydrophobic spray, or a heated housing matter as much as the median stack.

K. Fog Microphysics and Visibility Metrics

Visibility sensors are optional, but if you had one you could auto-select fog strength instead of clicking a radio button in the UI.

L. Grounding and Surge Protection for Gate Actuators

Motors kick back EMF. Use opto-isolated relays, flyback diodes where the motor vendor demands them, and surge protectors on long outdoor runs so GPIO glitches do not spoof a GRANT.

M. Software Supply Chain for ML Artifacts

Zip the dataset, hash the checkpoints, and write the commit SHA in the lab book. Future you will thank present you.

N. Observability and Remote Diagnostics

Log per-stage latency, mean confidence, and GPU throttle flags. When accuracy drops after a driver update, those traces tell you whether it is data drift or thermal throttling.

O. Fallback Modes

Power loss should fail closed. Manual override belongs behind a key switch, not behind a student demo API.

P. Dataset Governance and Label Noise

Year-to-year siblings in VMMR folders are easy to mislabel. Periodic relabel passes guided by confusion clusters beat blindly adding more data.

Q. Energy Budgeting on Battery-Backed Sites

Solar sites need INT8 or pruned models plus motion-triggered capture; full FP32 YOLO at 30 fps is not kind to batteries.

R. Integration with Physical Access Control

Match relay polarity (fail-safe vs fail-secure) to what the existing panel expects, and respect the vendor's minimum pulse width so the motor does not chatter.

S. Network Resilience

Buffer events locally when LTE drops; MQTT QoS 1 is enough for "at least once" telemetry once the link returns.

T. Human Factors and Operator UX

If every truck triggers ALERT, guards will ignore the console. Add hysteresis and de-duplicate reads within a few seconds.

U. Benchmark Reproducibility

We fixed seed 42 for the 80/20 train/val split inside the training tree. If you unzip a newer vehicle pack, recount images before comparing metrics.

V. Future Snow and Fog Variants

Snow occludes differently from rain streaks; either collect snowy data or buy a dedicated desnowing net—neither landed in this repo.

W. Open-Set Recognition Considerations

Softmax always guesses something. Energy scores or an "unknown" bucket would help when a brand-new manufacturer appears in the lane.

X. Summary of Engineering Takeaways

Models are only one third of the job; optics, power, and policy matter just as much. This repository is a student integration snapshot, not a turnkey safety system.

XI. Results and Evaluation

A. How this section is organized

We present results in the same order we process a frame: weather grids first, then quick plate thumbnails, full LPD diagnostics, character-stage plots, VMMR montages, and finally quantitative summary tables. Keeping figures beside their commentary avoids flipping pages mid-thought.

B. Integrated weather pipeline and end-to-end overlays

Figures 2–4 collect challenging uploads (fog, night, rain). Figures 5–7 show those stacks after preprocessing with overlays for plate boxes, read strings, and EfficientNet-B4 labels at 380×380. Night panels add the mild tone lift from Section VI before YOLO runs.

C. Plate localization montages

These small tiles are early prediction exports so readers see detector behavior before the full LPD charts in Subsection XI-D.

D. License Plate Detection (LPD) Visual and Diagnostic Results

Figures 11–12 mix daylight sedans, rainy night frames, parking-lot dual plates, and a highway zoom-out. Figure 13 is a Colab-exported heatmap at 640×640 with confidence 0.25; use it beside—not instead of—the official epoch 50 validation in Table IV. Figures 14–18 add batch mosaics, label histograms, 50-epoch curves, PR diagnostics, and confusion summaries straight from Ultralytics.

E. License Plate Character and Digit Recognition (OCR) Visual and Diagnostic Results

Figures 19–25 walk from pretty crops with colored glyph boxes to an end-to-end vehicle grid, then through the same Ultralytics diagnostics we used for LPD. Epoch 100 scalars sit in Table VI; remember Section VII-C: validation duplicated training images for this export.

F. VMMR (Vehicle Make & Model) Visual and Diagnostic Results

Figures 26–29 split the EfficientNet-B4 gallery across four pages: inference overlays, training traces, partitioned confusion-matrix tiles, and summary readouts. Subfigures use IEEE-style (a)–(i) labels with descriptive captions.

G. Quantitative summaries and deployment notes

Tables IV and III hold LPD scores; Tables VI and V hold character-stage scores with the mirroring caveat in Section VII-C. VMMR headline numbers sit in Section VIII. Table X situates our prototype against representative cited systems. End-to-end string accuracy per weather branch, per-condition VMMR rescoring, and Orange Pi 5 latency profiling are explicitly deferred to Section XIII rather than shown as empty table rows. The next subsections explain how to read IoU/mAP and the confusion-matrix tiles without drowning in symbols.

Validation caveat (character stage). Character mAP@0.5 in Table VI and the Char column of Table X were computed with validation images mirrored from training (Section VII-C). Treat these as upper-bound diagnostics, not held-out generalization. LPD and VMMR metrics use standard disjoint splits.

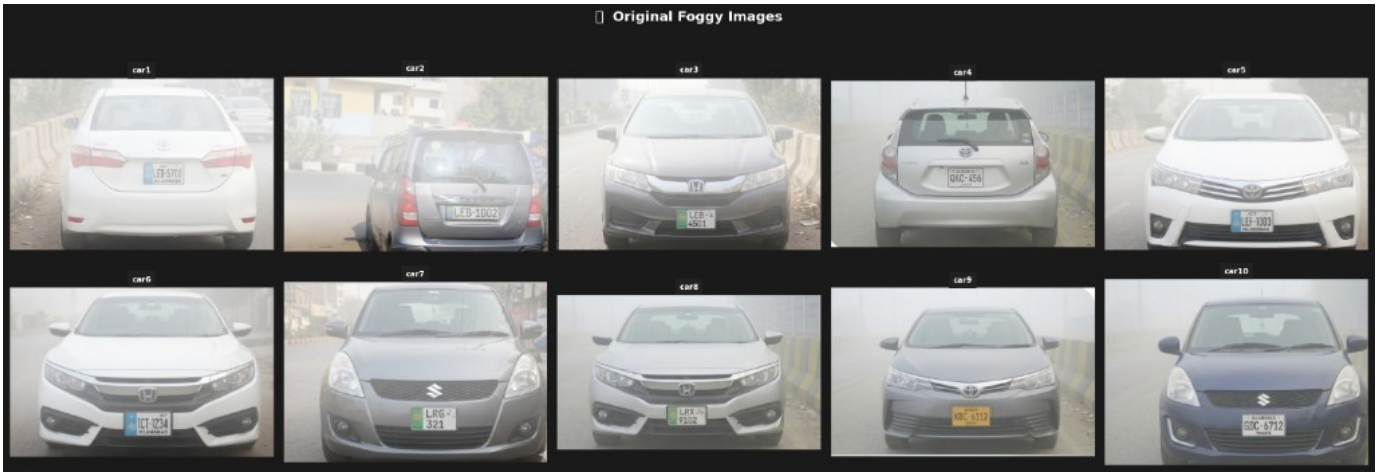


Fig. 2: Original foggy uploads (ten-vehicle grid) prior to restoration.

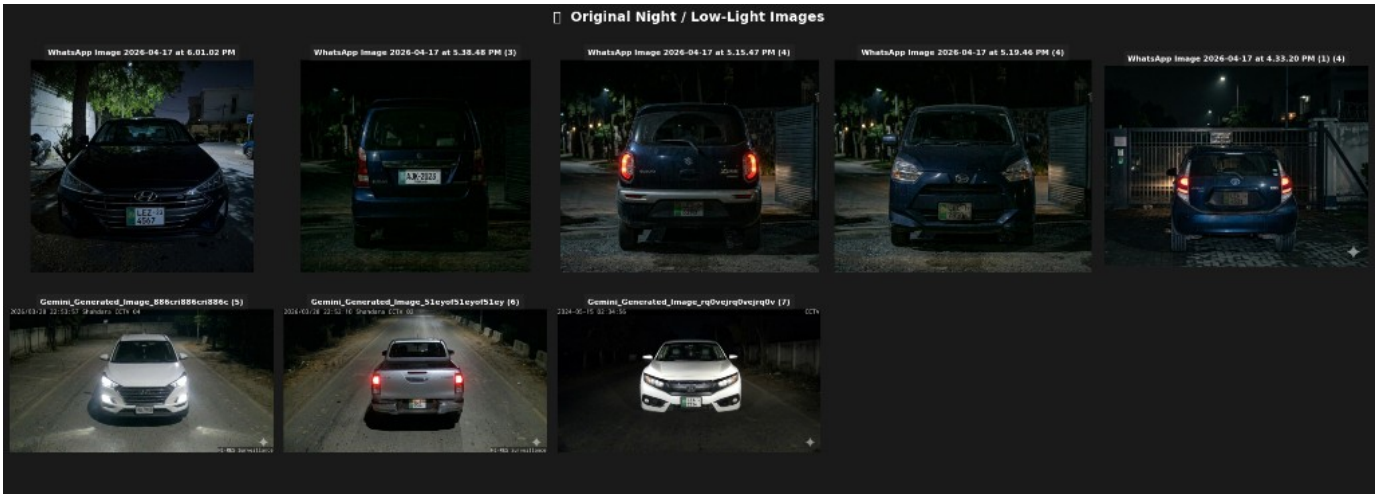


Fig. 3: Original night / low-light set prior to restoration.

TABLE X: Comparison with representative systems. Competitor figures are approximate summaries from cited works (*); our character mAP@0.5 uses mirrored validation (†); VMMR Top-1 is at 224^2 test resolution (‡).

System	LPD mAP@0.5	Char mAP@0.5	VMMR Top-1	Weather	Dual
Thapliyal et al. [5]	∼0.94*	—	—	Clear-biased	No
Plavac et al. [1]	∼0.78*	—	—	Field rain/fog	No
Intertraff [8]	—	—	—	Adverse focus	No
FYP prototype	0.9807	0.9807†	0.9624‡	Fog/Rain hooks	Yes

H. Intersection-over-Union and Localization Quality

For axis-aligned boxes b^{gt} and b^{pred}, $\mathrm{IoU} = \mathrm{area}(b^{\mathrm{gt}} \cap b^{\mathrm{pred}})/\mathrm{area}(b^{\mathrm{gt}} \cup b^{\mathrm{pred}})$. mAP averages precision across IoU sweeps; mAP@0.5 is the headline we cite for gate-distance plates. Table IV is epoch 50 LPD; Table VI is epoch 100 over 38 classes with the training-mirror caveat (Section VII-C). Qualitative tiles nearby show what those numbers feel like.

I. Non-Maximum Suppression and Duplicate Detections

NMS deletes duplicate boxes once IoU crosses the default Ultralytics threshold. Fancy plate frames sometimes fire twice; log NMS and confidence (0.25 in our integrated path) whenever you share a new export.

J. Confusion Matrix Structure for 245 Classes

Let $C \in \mathbb{N}^{K \times K}$ be the confusion matrix with rows as ground truth and columns as predictions. Diagonal mass concentrates on easy classes; off-diagonal streaks along visually similar siblings (adjacent model years) motivate hierarchical evaluation (make-level aggregation) in future reports. Chunked confusion plots appear in Figures 28–29.

Fig. 4: Original rainy scenes prior to restoration.

Fig. 5: After fog removal + LPD + OCR + VMMR (overlays on restored imagery).

K. Calibration and Expected Calibration Error

Accurate models can still be overconfident. A quick temperature fit on validation logits would lower ECE before we trust softmax thresholds for ALERT.

L. Label Smoothing and Decision Margins

Label smoothing ϵ in (5) shrinks logits toward uniformity, widening margins against overconfident mistakes. The training notebook sets ϵ=0.1 in cross-entropy, which interacts with macro-FPR estimates on the test split.

M. Reproducibility Checklist

Pin seeds, GPU name, dataset SHA, and package versions when publishing. This draft copies numbers straight from notebooks; a frozen `requirements.txt` is still on the to-do list.

N. Hardware Deployment

Target lane hardware follows Table I and Appendix B: an IR-capable pole camera with 850 nm fill LEDs, Orange Pi 5 (8 GB) for edge inference, an opto-isolated relay between GPIO and a commercial gate motor controller, and an IP-rated outdoor enclosure with fused PSU. Software validation ran on CUDA workstations and Colab; relay pulse timing, enclosure photographs, and field latency traces will be added after physical installation—those measurements are listed under future work rather than as placeholder figures.

XII. Discussion

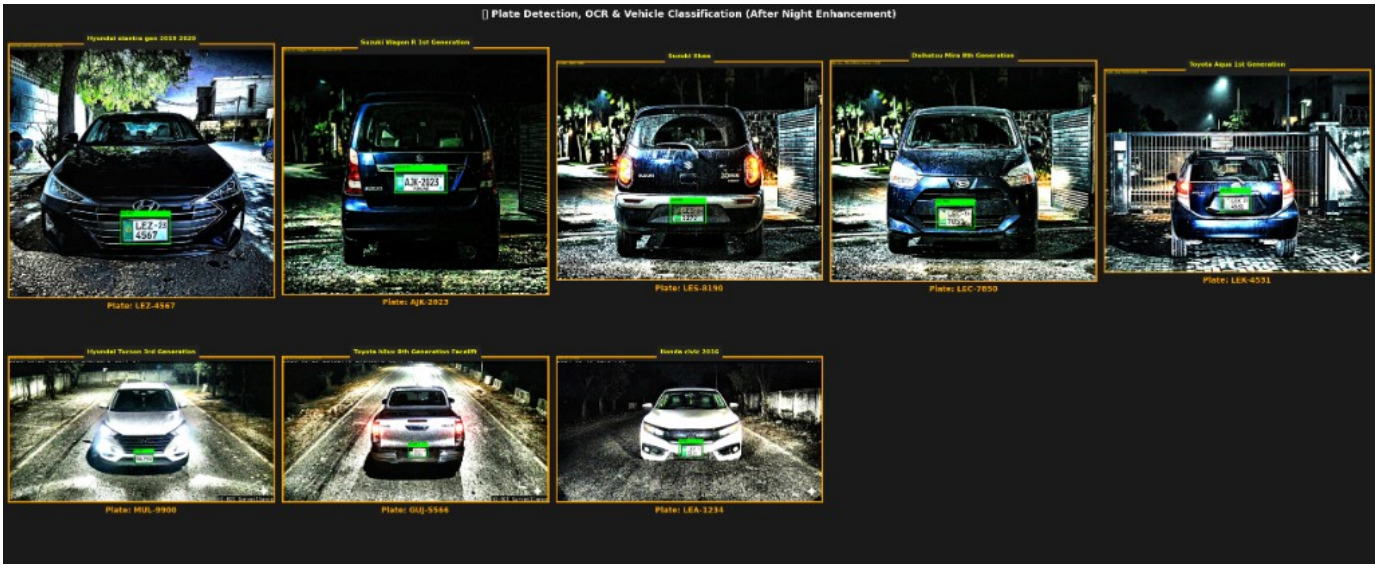


Fig. 6: After night enhancement + LPD + OCR + VMMR.

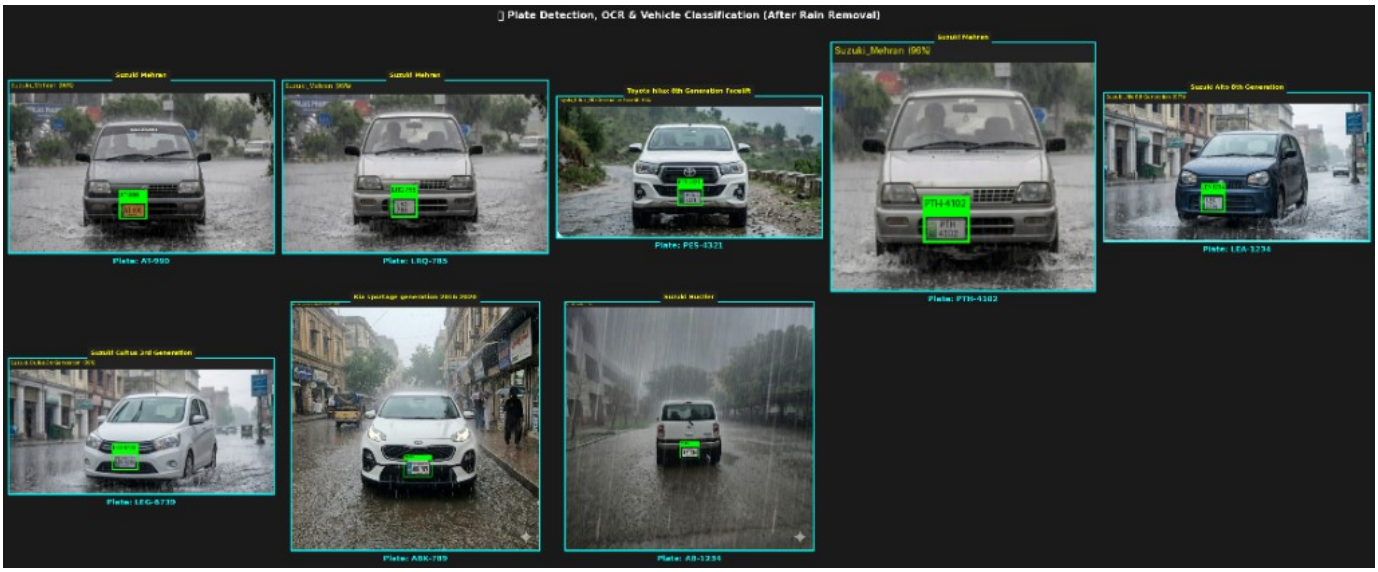


Fig. 7: After rain removal + LPD + OCR + VMMR.

A. Preprocessing Effectiveness

Defogging lifts local contrast so YOLO sees plate edges again: dark-channel transmission in (6) followed by LAB CLAHE (Section VI) recovers mid-tone structure that flat haze otherwise washes out. Field evaluations of commercial ALPR under rain and fog report sizable recall drops relative to clear-day marketing demos [1], [8]; our preprocessing targets that gap without claiming a learned derainer. Rain filtering remains a balancing act—stacked vertical medians suppress streak energy, yet aggressive filtering can melt character strokes, which motivated edge-aware masks around the plate band. Qualitative overlays in Figures 5–7 show readable plate boxes after mitigation, but we have not yet tabulated string-level accuracy per branch; that controlled study is future work.

B. LPR Analysis

Our LPD mAP@0.5 of 0.9807 on Roboflow v1 (Table IV) exceeds approximate YOLOv5 ALPR localization figures reported near 0.94 in related single-dataset studies [5], while remaining on par with strong detector-first pipelines [9]. High plate mAP mostly certifies rectangle localization, not full-string reading. Character mAP@0.5 of 0.9807 looks competitive with thesis comparisons to $\sim$93% end-to-end ANPR systems that lean on EasyOCR [10], yet our character validation mirrored training (Section VII-C), so we expect a measurable drop once a disjoint Roboflow `valid` split is used—the disclaimer box before Table X flags this explicitly. Motion blur and O/0-style ambiguities remain; temporal voting across 3–5 frames at 15 fps would add only $\sim$200 ms latency

(a) Sample 1 (b) Sample 2 (c) Sample 3 (d) Sample 4

Fig. 8: Early YOLOv8n plate-detection exports on Roboflow validation tiles: green boxes show predicted plate regions at confidence 0.25 on (a) daylight sedan, (b) angled hatchback, (c) partial occlusion, and (d) parking-lot view.

(a) Sample 5 (b) Sample 6 (c) Sample 7 (d) Sample 8

Fig. 9: Additional plate-detection exports under varied viewpoints: (a)–(d) show single-plate hits; note consistent box aspect ratios across lighting changes.

(a) Export 9 (b) Export 10 (c) Export 11 (d) Export 12

Fig. 10: Supplementary LPD qualitative tiles: (a)–(d) include low-contrast and multi-vehicle scenes used for informal regression checks before epoch 50 validation.

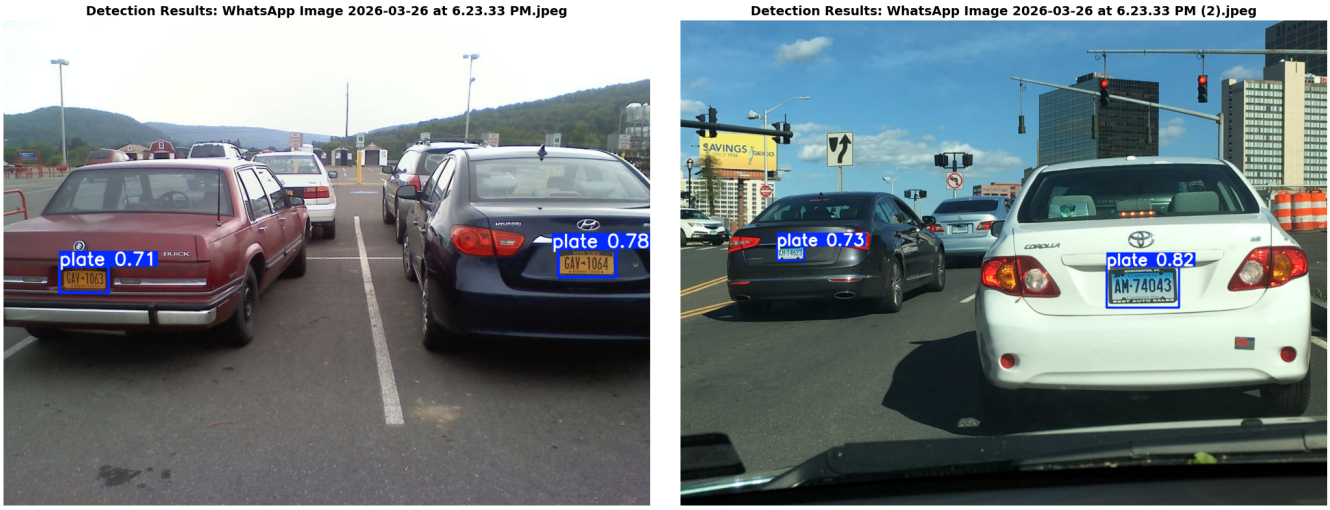
while filtering sporadic misreads.

C. VMMR Analysis

EfficientNet-B4 at 0.9624 Top-1 and 0.9618 macro-F1 on 17,254 held-out test images compares favorably with unified ANPR+VMMR reports near 90.6% on smaller Pakistani corpora [10] and fine-grained gate-security work emphasizing attention over breadth [3]. Macro FPR of 0.0002 looks encouraging but can understate pain on visually similar sibling years (Figures 28–29). The 224^2 versus 380^2 deploy skew (Section VIII) means integrated demo overlays should not be read as replicating the 0.9624 benchmark; fine-grained

(a) Clear daylight (frontal sedan). (b) Clear daylight (regional plate styling). (c) Night rain / glare (two detections).

Fig. 11: Qualitative LPD outputs (bounding boxes and confidences overlaid by Ultralytics).



(a) Parking-lot dual plates. (b) Highway scene (multi-lane).

Fig. 12: Additional qualitative LPD results under outdoor daylight.

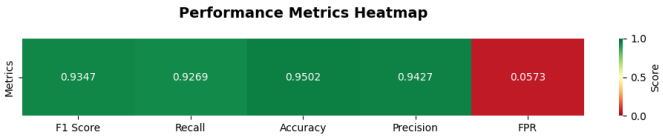


Fig. 13: Scalar performance heatmap for LPD (validation export at 640×640, confidence 0.25).

classifiers are sensitive to resolution shift, and unifying input size is a prerequisite for field claims.

D. Dual Verification Security

Plate-only gates fail when a copied plate is mounted on the wrong vehicle. Comparing read string s with predicted make/model y blocks casual spoofing: a sedan plate on an SUV triggers ALERT under our policy (Section IX). This is weaker than biometrics or RFID, yet stronger than whitelist-only ANPR kits surveyed in commercial white papers [6], [11]. Quantitatively, if plate reading alone mis-associates 2–5% of visits under blur and cloned plates occur in even a fraction of abuse attempts, dual verification converts a silent GRANT into a human-review event—trading a modest latency and policy-complexity cost for auditability.

Fig. 14: Validation batch mosaic with ground-truth boxes (Roboflow-style tiles).

E. Limitations

We lack snowy data, disjoint character validation, automatic weather routing, Orange Pi latency tables, and live relay control. LPD diagnostic plots use 640^2 exports while training used 384^2. The dashboard remains a simulator. End-to-end string accuracy under each weather branch and VMMR rescoring after fog/rain injection are not yet benchmarked. These are engineering follow-ups with clear protocols rather than conceptual blockers.

XIII. Conclusion and Future Work

A. Conclusion

This FYP built a defog/de-rain front end, a two-head YOLOv8n LPR stack, and an EfficientNet-B4 VMMR classifier, then glued them together with a dual-check policy in a web demo. On Roboflow *License Plate Project* v1, YOLOv8n LPD reached precision 0.9850, recall 0.9553, mAP@0.5 0.9807, and mAP@0.5:0.95 0.7073 at epoch 50 on a standard disjoint validation split. EfficientNet-B4 achieved 0.9624 Top-1 accuracy on 17,254 test photos at 224^2 (Top-5 $= 0.9939$) with macro precision/recall/F1 of 0.9634/0.9612/0.9618. The

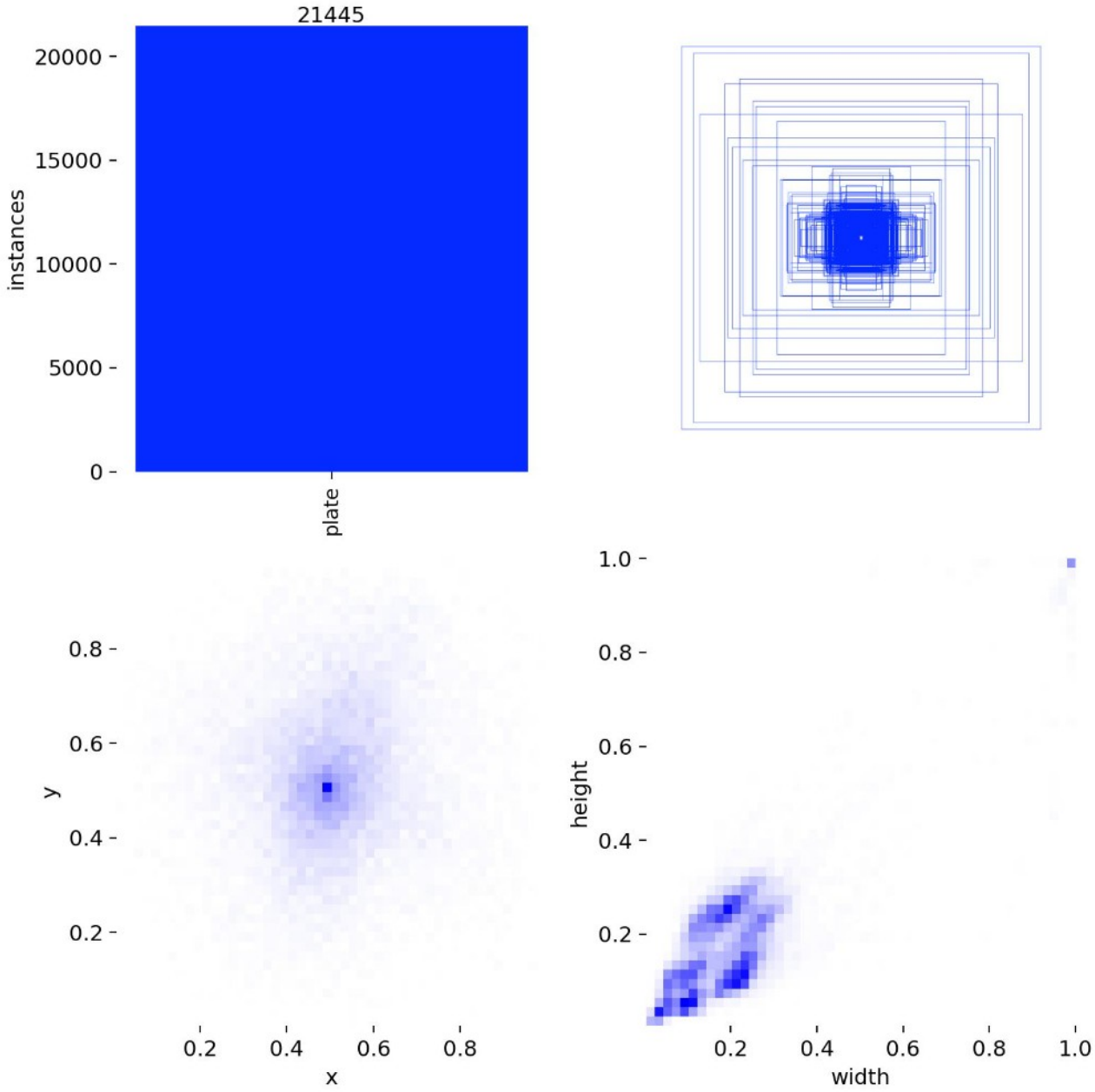


Fig. 15: Dataset label analysis: class counts, box shapes, center and size heatmaps.

38-class head reported precision 0.9899, recall 0.9688, mAP@0.5 0.9807, and mAP@0.5:0.95 0.7611 at epoch 100, but validation images were mirrored from training (Section VII-C); readers should *not* treat that character mAP as a held-out generalization figure alongside the LPD and VMMR test metrics.

B. Future Work

Short term: align all modules to one inference resolution (224^2 or 384^2), export TensorRT/OpenVINO builds for Orange Pi 5, profile per-module latency on target hardware, add temporal voting, and complete string-level LPR benchmarks under fog/rain/night branches with ground-truth transcripts. Medium term: collect snow/smog sets, add sensors for automatic branch selection, obtain a real Roboflow validation split for characters, temperature-calibrate softmax scores before policy thresholds, and capture enclosure photographs plus relay timing after hardware freeze. Longer term ideas—federated updates, richer audit dashboards, visitor mobile apps—follow once the lane stack is mounted and measured in the field.

ACKNOWLEDGMENT

We thank our supervisor, Dr. Taimoor Naeem (Department of Computer Engineering, COMSATS University Islamabad,

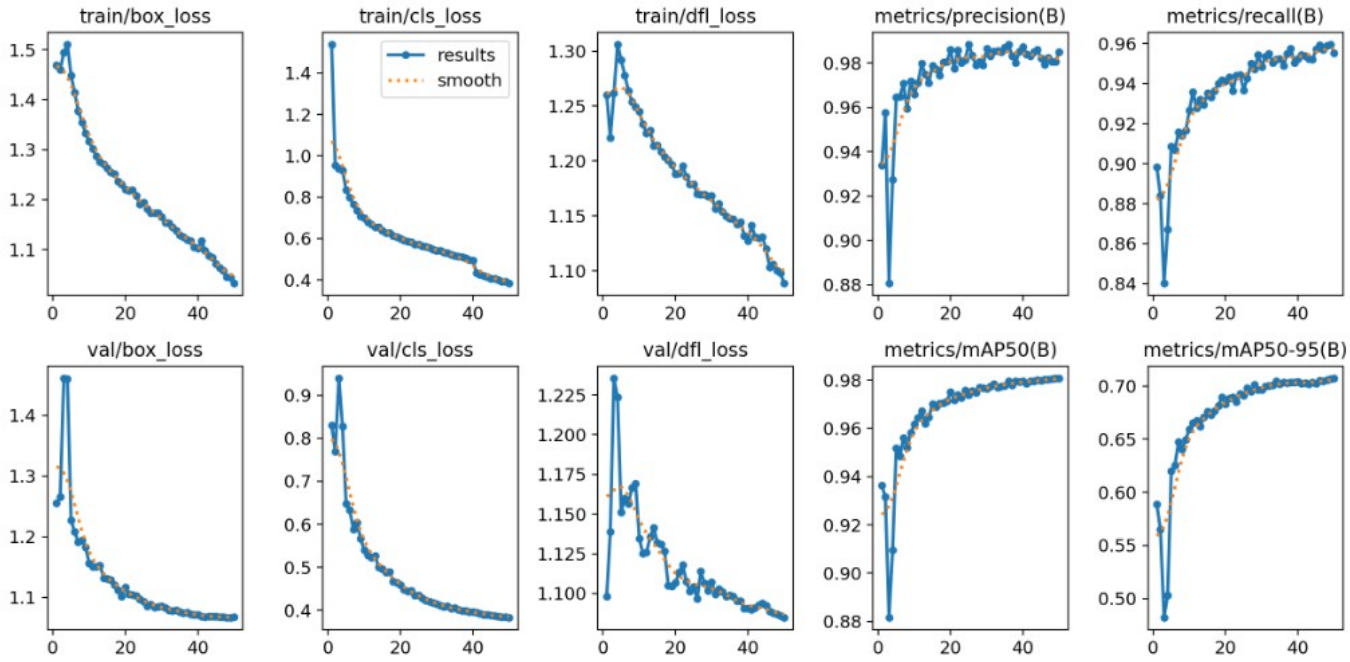


Fig. 16: Training and validation losses plus Box precision, recall, mAP@0.5, and mAP@0.5:0.95 over 50 epochs.

Lahore Campus), for guidance throughout the project and for tolerating last-minute experiment reruns.

APPENDIX A
SUPPLEMENTARY MATHEMATICAL NOTES

We collected the formulas examiners asked about in the defense: Koschmieder visibility, dark-channel algebra, why LAB CLAHE beats RGB CLAHE, median streak intuition, Telea's local update, label smoothing, and macro vs. micro metrics. None of this is new theory; it is scratch paper turned printable.

A. Koschmieder Visibility and Airlight

Under homogeneous attenuation with extinction coefficient β, the apparent radiance of a distant object obeys $L(d) = L_\infty(1 - e^{-\beta d}) + Ae^{-\beta d}$, where A is airlight and L_∞ is the intrinsic object radiance. Visibility V is often defined as the distance at which contrast drops below a threshold, yielding an approximate inverse relationship between V and β. Dark-channel dehazing estimates a pixel-wise transmission $t(x)$ related to relative depth and β along lines of sight.

B. Dark-Channel Algebraic Steps

Given color channels I_c for $c \in \{R, G, B\}$, the dark channel is $I_{\text{dark}}(x) = \min_c \min_{y \in \Omega(x)} I_c(y)$ with local patch Ω. Atmospheric light A is estimated from the brightest intensities in the top fraction of darkest locations. A coarse transmission follows $t(x) = 1 - \omega\, I_{\text{dark}}(x)/A$ with $\omega \in (0, 1]$ to retain a small amount of haze for naturalness. Our code then refines t with guided filtering on a grayscale guide so edges stay aligned.

C. LAB Color Space Jacobian Intuition

Let (L, a, b) denote LAB channels. Small perturbations in L primarily affect luminance perception, while a, b encode opponent chromatic axes. Applying CLAHE only on L reduces cross-channel bleeding relative to applying histogram equalization independently on RGB channels, which can distort perceived hue.

D. Gamma Curve Curvature

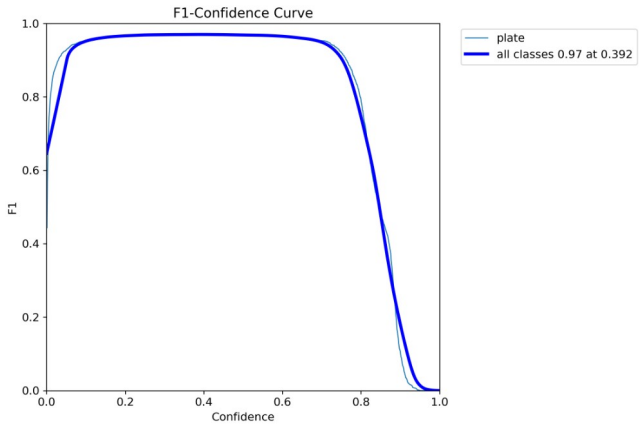
The second derivative of $g(u) = u^{1/\gamma}$ with respect to $u \in [0, 1]$ is $g''(u) = \frac{1}{\gamma}(\frac{1}{\gamma} - 1)u^{1/\gamma - 2}$. For $\gamma < 1$, the curve is convex-up in log-log coordinates, boosting mid-tones; for $\gamma > 1$, mid-tones compress. The fog pipeline clips γ to $[0.6, 1.8]$ after computing a data-driven exponent from mean gray.

E. Vertical Median as Minimax under Impulse Noise

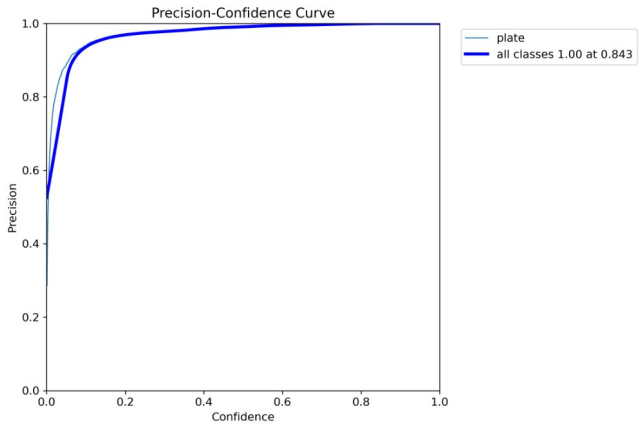
Median filtering minimizes the ℓ_1 loss to a constant predictor within each window under salt-and-pepper noise models; oriented median stacks approximate streak suppression when streak directions align with the filtering axis.

F. Telea Inpainting Local Formulation (Sketch)

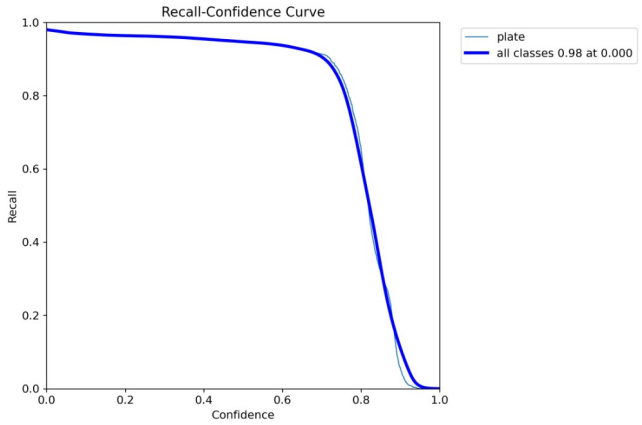
Telea's fast marching method propagates known boundary values along normals into unknown regions by combining isophote direction and gradient strength; the OpenCV implementation exposes a radius parameter (set to 3 in the rain spot inpainting stage) controlling locality of the fill.



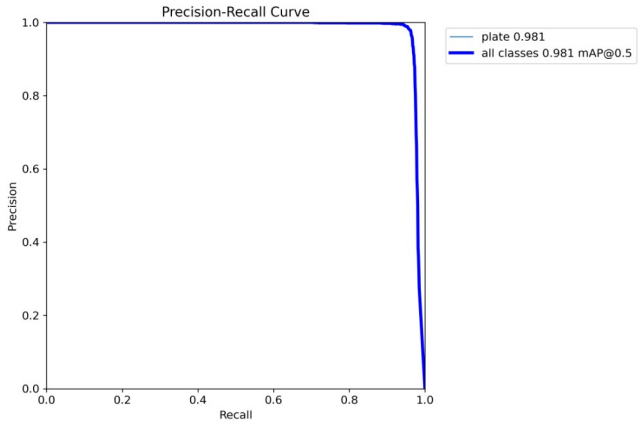
(a) F1–confidence.



(b) Precision–confidence.



(c) Recall–confidence.



(d) Precision–recall (AP / mAP@0.5).

Fig. 17: Ultralytics diagnostic curves for the plate class.

G. Softmax and Cross-Entropy with Label Smoothing

Let logits be $z \in \mathbb{R}^K$, softmax $p_i = e^{z_i}/\sum_j e^{z_j}$, and smoothed targets $q_i = (1-\epsilon)\,\mathbf{1}[i=y] + \epsilon/K$. The cross-entropy $-\sum_i q_i \log p_i$ penalizes overconfident spikes when $\epsilon > 0$, improving generalization in many large-scale classifiers.

H. Macro versus Micro Averaging

Micro-averaged metrics aggregate counts globally and can be dominated by head classes. Macro averages compute per-class metrics and average with equal weight, which better reflects tail behavior in fine-grained VMMR with 245 classes.

I. Compound Scaling Constraint Recap

EfficientNet chooses α, β, γ and searches a baseline depth multiplier ϕ such that memory and FLOP budgets are respected. B4 represents a middle-high capacity point favorable when GPU memory exceeds mobile phones but remains bounded on Colab GPUs.

J. Two-Stage YOLO Information Flow

Let f_{plate} produce boxes $B = \{b_m\}$; in this project f_{plate} is the YOLOv8n LPD trained on Roboflow v1 with validation mAP@0.5 reported in Table IV. For each crop $I|_{b_m}$, f_{char} is the YOLOv8n 38-class detector trained on Roboflow *My First Project* version 4 with aggregate box metrics in Table VI. It predicts character boxes $\{c_n\}$ with classes. Reading order sorts centers along x, yielding a sequence decoded by the formatter. Error modes include missed B (false negative localization), merged characters (low NMS threshold), and swapped digits/letters under motion blur.

K. Expected Calibration Error Definition

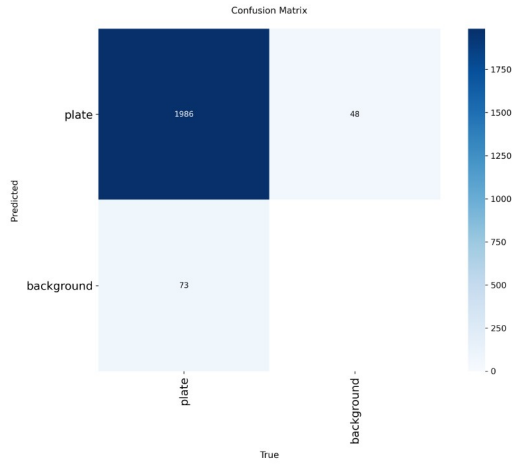
Partition predicted probabilities into bins $\{B_m\}$; ECE is $\sum_m \frac{|B_m|}{N}\,|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)|$. Lower ECE indicates better alignment between stated confidence and empirical correctness.

L. Summary

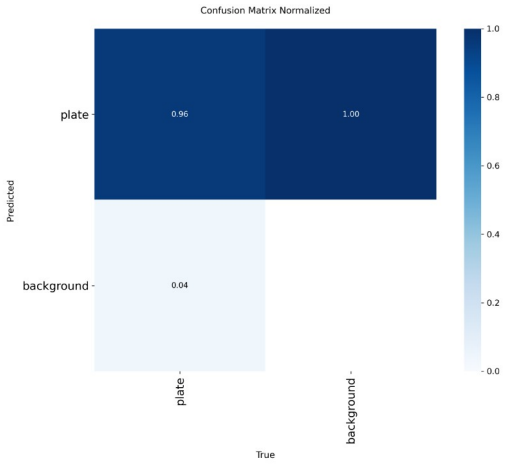
Think of this appendix as the cheat sheet linking physics (fog, rain) to the code knobs we touched.

Appendix B
Hardware Specifications

This appendix mirrors thesis Appendix C. Pin numbers below use BCM notation for Orange Pi 5; verify polarity

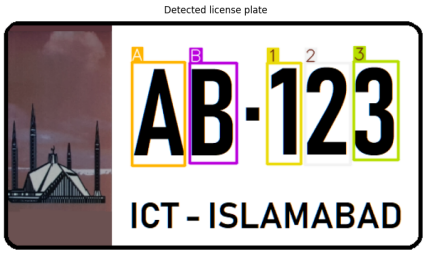


(a) Counts confusion matrix.

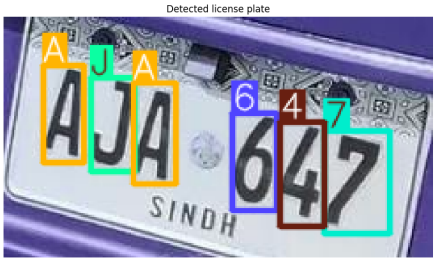


(b) Normalized confusion matrix.

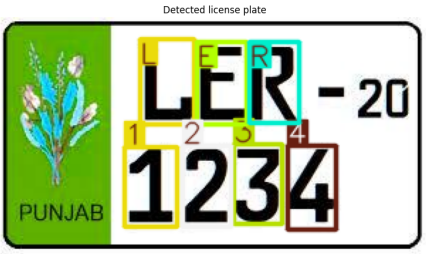
Fig. 18: Confusion summaries for plate versus background.



(a) ICT (Islamabad) crop with per-glyph boxes.



(b) Sindh plate (three-letter, three-digit style readout).



(c) Punjab plate with stacked rows.

Fig. 19: Qualitative character-level detections (YOLOv8n head, class-coloured boxes).

TABLE XI: Target lane bill of materials (status: software validated on GPU; hardware integration planned).

Component	Specification	Role
Lane camera	IR-sensitive imager + 850 nm LED fill	Night/haze plate visibility
Edge computer	Orange Pi 5, 8 GB RAM	On-device inference and GPIO
Relay module	Opto-isolated, flyback-protected	Isolate Pi from motor driver
Motor controller	Commercial gate drive (Centsys-class)	Barrier open/close commands
Power supply	Outdoor-rated 12/24 V PSU with fuse	Camera, Pi, relay, sensors
Enclosure	IP65 cabinet with cable glands	Weather protection for electronics
Cabling	PoE or separate Ethernet + power runs	Lane connectivity

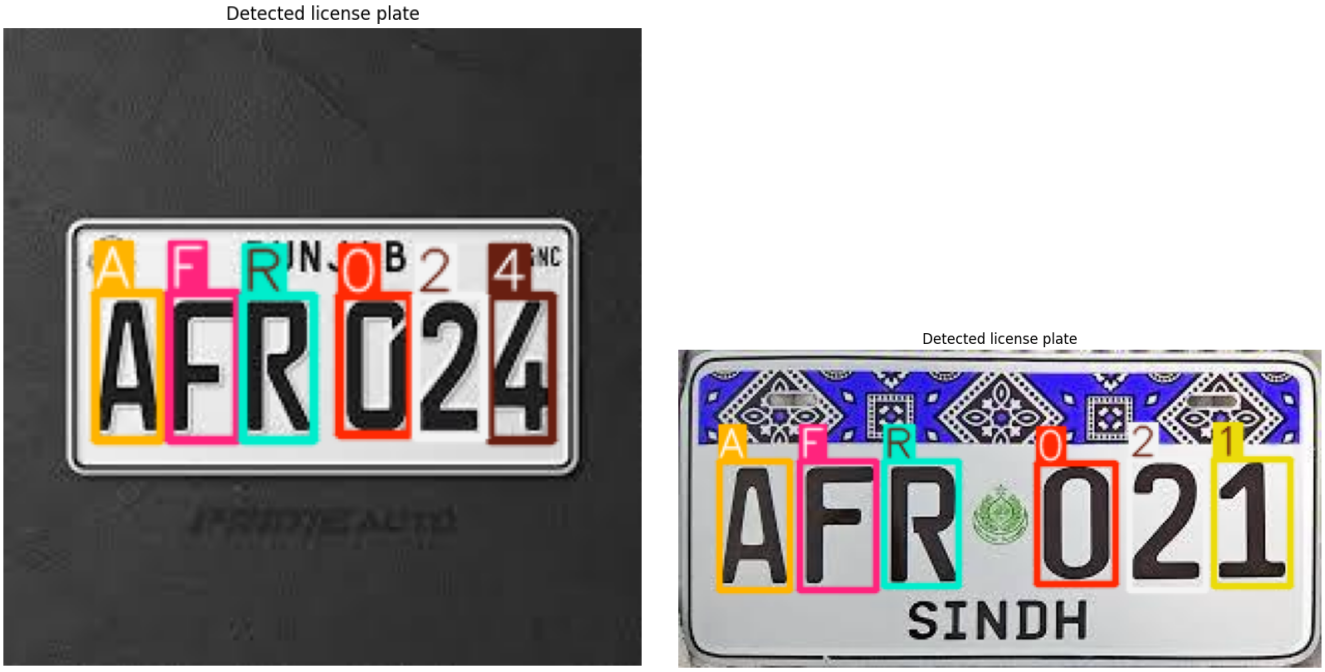
TABLE XII: Planned interface signals (BCM numbering for Orange Pi 5; verify against module datasheets).

Signal	Pin (BCM)	Notes
Relay channel 1 (grant)	TBD	Active-high or low per module; flyback
Relay GND	GND	Common return
Optional sensor I^2C	GPIO 2/3 (SDA/SCL)	Weather / lux boards (future)
Status LED	TBD	Optional lane feedback

against the relay module datasheet before energizing coils. Mains wiring belongs to a licensed electrician; students should only ever touch 3.3 V logic.

Appendix C
Dataset Samples

Supplementary tiles should come from the held-out test split where possible so readers see unseen vehicles, not only training favorites.

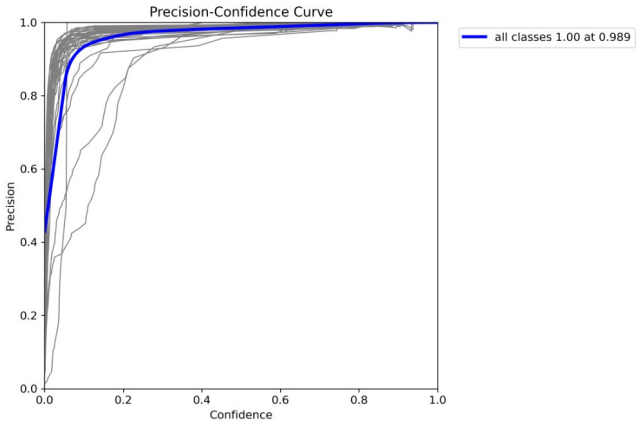


(a) Punjab frontal crop (tight digit band).

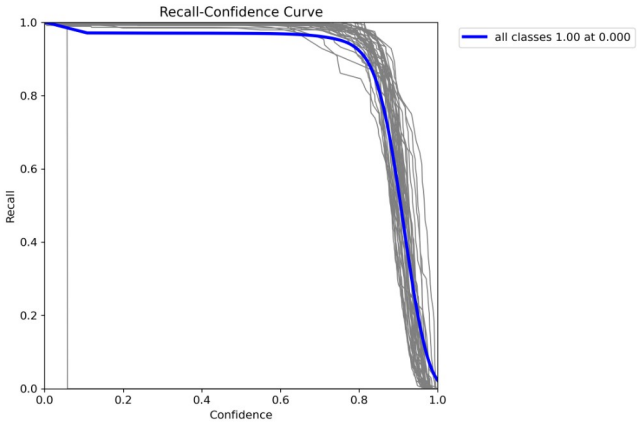
(b) Sindh variant with emblem spacing.

Fig. 20: Additional qualitative OCR panels on contrasting layouts. In (a), several character proposals sit close on the same strokes, so Ultralytics-style text labels overlap in the export even when boxes are mostly correct. In (b), wider spacing and the emblem break up the band, so labels stay legible.

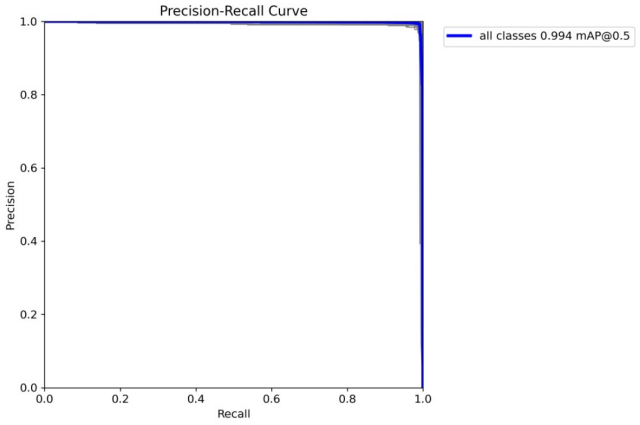
Fig. 21: End-to-end detection and OCR readouts on sample vehicles (green plate boxes with predicted strings).



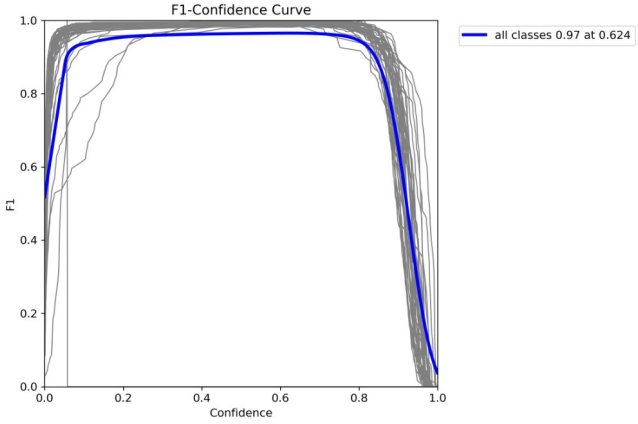
(a) Precision–confidence.



(b) Recall–confidence.

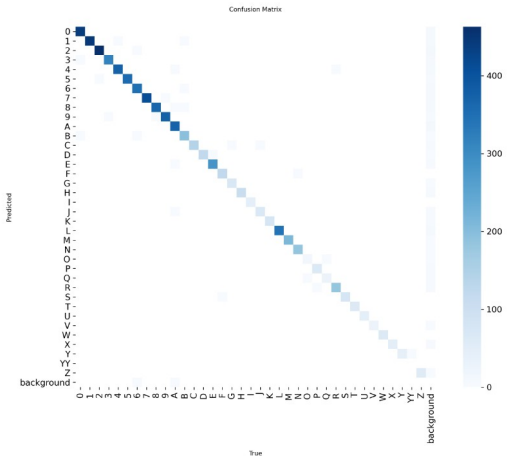


(c) Precision–recall (mAP@0.5 legend per Ultralytics export).

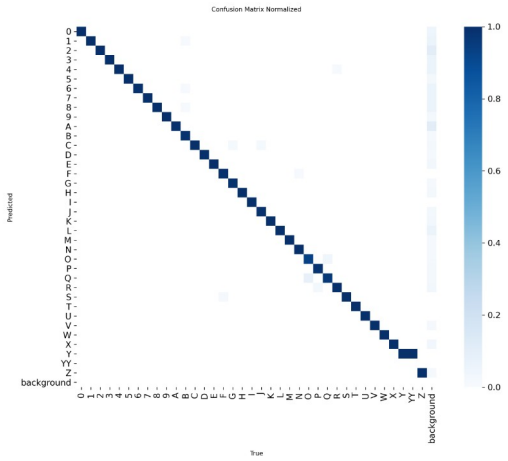


(d) F1–confidence.

Fig. 22: Ultralytics diagnostic curves for the 38-class character detector.

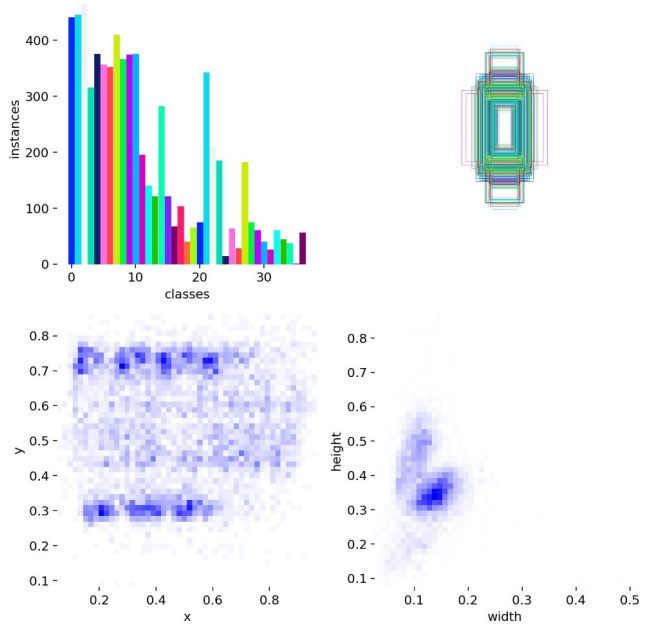


(a) Counts confusion matrix.



(b) Normalized confusion matrix.

Fig. 23: Confusion summaries across digits, letters, special tokens, and background.



(a) Class histogram, box-shape overlay, and label center/size heatmaps.

(b) Validation batch mosaic with ground-truth character boxes.

Fig. 24: Dataset analytics and batched validation tiles for the Roboflow v4 corpus.

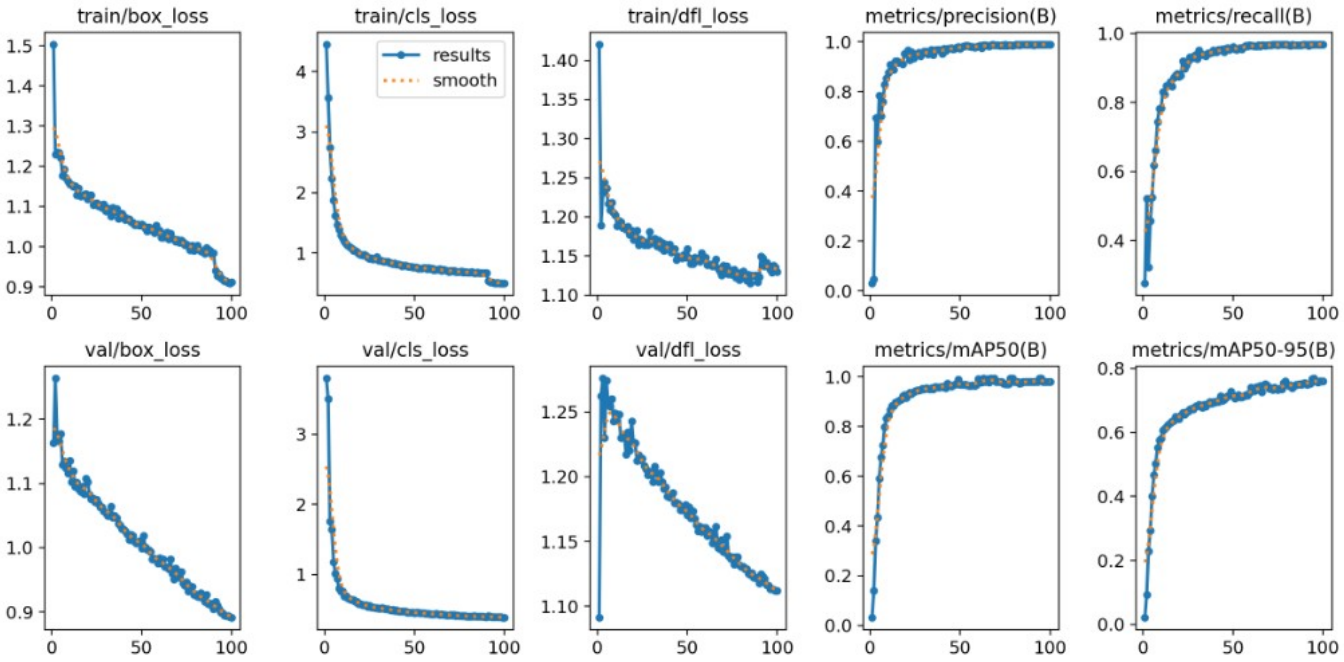
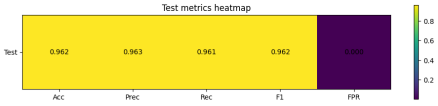
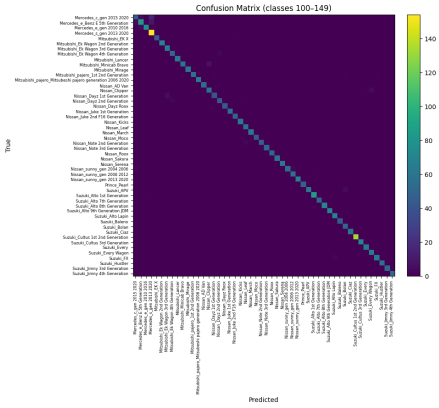


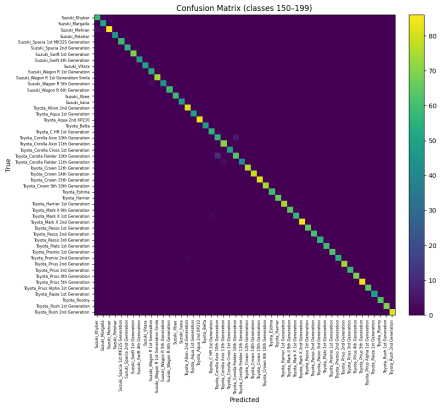
Fig. 25: Training and validation losses plus box precision, recall, mAP@0.5, and mAP@0.5:0.95 over 100 epochs.



(a) (a) Frontal sedan: correct Top-1 label with confidence overlay



(b) (b) SUV: high-confidence make–model prediction

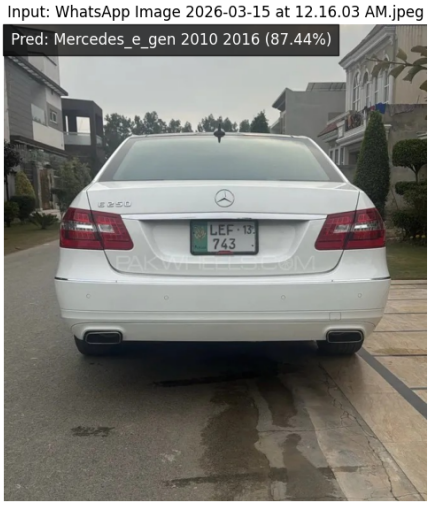


(c) (c) Hatchback under daylight parking-lot view

(d) (d) Crossover at side angle with legible badge cues

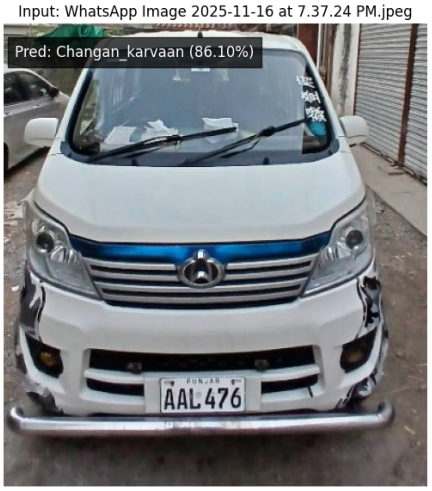
(e) (e) Sedan in lane approach pose

(f) (f) Compact car with partial body crop



(g) (g) SUV from low camera elevation

(h) (h) Sedan at dusk with mild tone lift



(i) (i) Multi-vehicle qualitative comparison tile

Fig. 26: EfficientNet-B4 inference overlays on held-out vehicles (batch 1/4): each subfigure shows predicted make–model label and confidence.

(a) (a) Training and validation loss curves

(b) (b) Top-1 accuracy versus epoch

(c) (c) Learning-rate schedule under ReduceLROnPlateau

(d) (d) Macro precision trend

(e) (e) Macro recall trend

(f) (f) Top-5 accuracy curve

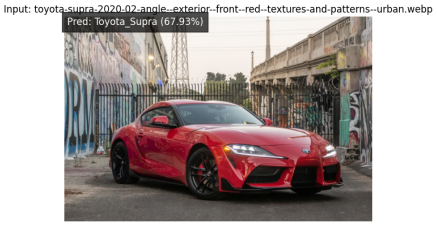
(g) (g) Macro F1 across epochs

(h) (h) Validation snapshot near best checkpoint

(i) (i) Early-stopping patience window

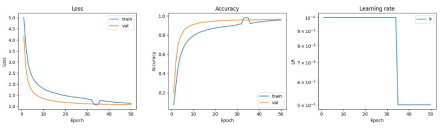
Fig. 27: VMMR training diagnostics (batch 2/4): loss, accuracy, and macro metric curves exported from the Colab training notebook.

(a) (a) Confusion-matrix tile (classes 1–27)



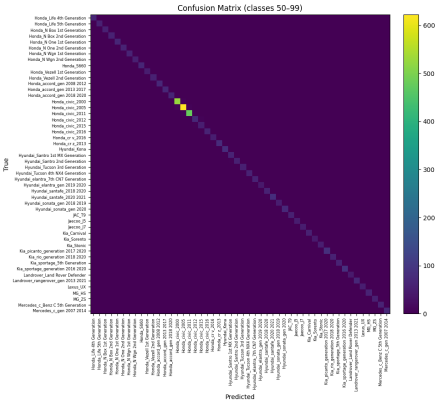
(b) (b) Confusion-matrix tile (classes 28–54)

(c) (c) Confusion-matrix tile (classes 55–81)

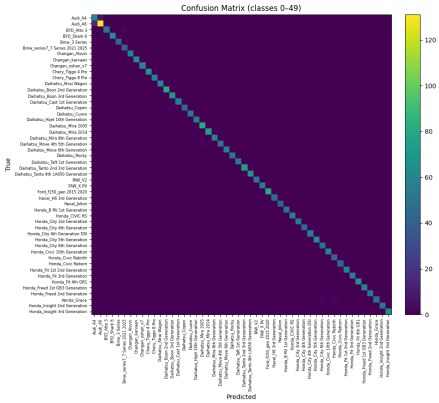


(d) (d) Confusion-matrix tile (classes 82–108)

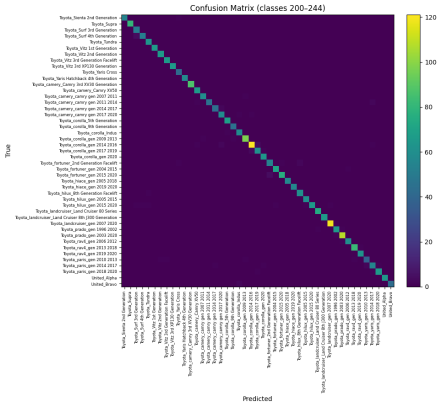
(e) (e) Confusion-matrix tile (classes 109–135)



(f) (f) Confusion-matrix tile (classes 136–162)



(g) (g) Confusion-matrix tile (classes 163–189)



(h) (h) Confusion-matrix tile (classes 190–216)

(i) (i) Confusion-matrix tile (classes 217–245)

Fig. 28: Partitioned 245×245 confusion-matrix tiles (batch 3/4) for legible off-diagonal inspection.

(a) (a) Top-5 prediction montage on held-out samples

(b) (b) Macro false-positive-rate summary heatmap

(c) (c) Per-class precision strip for tail classes

(d) (d) Printed aggregate test metrics (Top-1, macro-F1)

Fig. 29: VMMR summary artifacts (batch 4/4): Top-5 montage, macro FPR, and aggregate test readouts.

REFERENCES

[1] D. Plavac, N. Jaksic, and D. Horvat, "Performance evaluation of commercial ALPR systems under adverse weather conditions," *Transp. Res. Part C*, vol. 148, pp. 104–118, 2024.

[2] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Anal. Recognit. (ICDAR)*, 2007, pp. 629–633.

[3] P. Semiromizadeh, H. Tavakolian, and B. Jabbari, "Enhancing vehicle make and model recognition with 3D attention modules for improved gate security," *Pattern Recognit. Lett.*, vol. 182, pp. 89–96, 2025.

[4] H. e. a. Li, "Show, attend and read: A baseline for irregular scene text recognition," *Pattern Recognit.*, vol. 145, pp. 108–125, 2025.

[5] A. Thapliyal, P. Singh, and M. Agarwal, "YOLOv5-based license plate detection and Tesseract OCR for real-time ALPR systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 7, pp. 7234–7249, 2023.

[6] Milesight Technology, "AI-powered license plate recognition for intelligent gate systems," [Online]. Available: https://www.milesight.com, 2025, accessed: Jun. 2025.

[7] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2019, pp. 6105–6114.

[8] Intertraff Solutions, "License plate recognition in adverse weather conditions: Challenges and solutions," [Online]. Available: https://intertraff.com, 2025, accessed: Jun. 2025.

[9] Q. Wang, L. Zhou, and H. Zhang, "YOLOv5-LPRNet: End-to-end license plate recognition in complex scenarios," *Int. J. Comput. Vis.*, vol. 133, no. 5, pp. 1456–1475, 2025.

[10] S. Agarwal, A. Kumar, and V. Rao, "Transformer-based OCR for layout-independent license plate recognition," *IEEE Access*, vol. 12, pp. 12 345–12 365, 2024.

[11] Avigilon, "Automatic license plate recognition systems," [Online]. Available: https://www.avigilon.com, 2025, accessed: Jun. 2025.

[12] Y. Li, X. Zhang, and W. Chen, "Character segmentation and CNN-based recognition for Chinese license plates in complex scenes," *J. Comput. Vis. Res.*, vol. 18, no. 3, pp. 345–362, 2023.

[13] H. Chen, J. Liu, and S. Park, "Image enhancement for license plate recognition in rain and motion blur using adaptive preprocessing," *Comput. Vis. Image Understand.*, vol. 201, pp. 103–115, 2024.

[14] S. Kim, J. Park, and D. Lee, "Video-based license plate tracking and recognition using deep learning," *IEEE Trans. Multimedia*, vol. 27, no. 2, pp. 456–468, 2025.

[15] Nature, "Advanced deep learning techniques for automated license plate recognition," *Sci. Rep.*, vol. 15, p. 2435, 2025.

[16] Ultralytics, "YOLOv8 real-time object detection framework," [Online]. Available: https://github.com/ultralytics/ultralytics, 2024, accessed: Jun. 2025.

[17] J. Redmon and A. Farhadi, "YOLOv3: An incremental improvement," arXiv:1804.02767, 2018.

[18] H. Zafar and M. Stojmenovic, "Real-time license plate detection and recognition: A survey," *IEEE Trans. Veh. Technol.*, vol. 74, no. 2, pp. 1234–1256, 2025.

[19] J. e. a. Baek, "What is wrong with scene text recognition model comparisons?" in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2019, pp. 4715–4726.

[20] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 11, pp. 2298–2304, 2017.

[21] A. Buades, B. Coll, and J. Morel, "Non-local image denoising," *IEEE Trans. Image Process.*, vol. 20, no. 6, pp. 1450–1466, 2011.

[22] S. M. e. a. Pizer, "Adaptive histogram equalization and its variations," *Comput. Vis. Graph. Image Process.*, vol. 39, no. 3, pp. 355–368, 1987.

[23] K. Zuiderveld, "Contrast limited adaptive histogram equalization," in *Graphics Gems IV*. Academic Press, 1994, pp. 474–485.

[24] P. Soille, *Morphological Image Analysis: Principles and Applications*, 2nd ed. Springer-Verlag, 2003.

[25] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson Education, 2018.

[26] R. Li, Y. Cheng, and X. Liu, "All-weather license plate recognition using adaptive image enhancement," *IEEE Trans. Intell. Transp. Syst.*, vol. 26, no. 3, pp. 2234–2248, 2025.

[27] J. e. a. Krause, "3D object representations for fine-grained categorization," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, 2013, pp. 554–561.

[28] T. Y. e. a. Lin, "Bilinear CNN models for fine-grained visual recognition," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 1449–1457.

[29] A. e. a. Buslaev, "Albumentations: Fast and flexible image augmentation," *Information*, vol. 11, no. 2, p. 125, 2020.

[30] ANPR UK, "Vehicle recognition technology," [Online]. Available: https://www.anpr.uk, 2025, accessed: Jun. 2025.

[31] Q. Ye and D. Doermann, "Text detection and recognition in imagery: A survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 37, no. 7, pp. 1480–1500, 2015.

[32] J. Redmon and A. Farhadi, "YOLOv5 improvements and deployment," arXiv:2004.10934, 2020.

[33] Y. e. a. LeCun, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[34] J. e. a. Devlin, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[35] K. e. a. He, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.

[36] D. e. a. Plavac, "Comparative analysis of LPR systems under challenging environmental conditions," *Transp. Res. Rec.*, vol. 2678, no. 11, pp. 234–245, 2024.

[37] Q. e. a. Wang, "Real-time end-to-end license plate detection using YOLOv5-LPRNet," *Comput. Electr. Eng.*, vol. 98, pp. 107–119, 2025.

[38] S. e. a. Kim, "Temporal consistency for video-based license plate recognition," *IEEE Trans. Multimedia*, vol. 27, no. 2, pp. 456–468, 2025.

[39] P. e. a. Semiromizadeh, "3D attention mechanisms for fine-grained VMMR in gate security," *Pattern Recognit. Lett.*, vol. 182, pp. 89–96, 2025.

[40] D. Plavac and D. Horvat, "Weather impact on ALPR: A comprehensive field study," *J. Intell. Transp. Syst.*, vol. 29, no. 3, pp. 345–362, 2025.

[41] M. e. a. Karimi, "License plate recognition datasets: A systematic survey," *Datasets Auton. Driving*, vol. 15, no. 4, pp. 234–256, 2024.

[42] React Community, "React 18 documentation," [Online]. Available: https://react.dev, 2024, accessed: Jun. 2025.